Fine-Tuning Transformer Models for Question Answering

1. Introduction

Question Answering (QA) tasks differ from traditional classification tasks in how predictions are made. In classification, the model assigns a fixed label (such as positive or negative) to an input. However, in QA, the model identifies a relevant span of text from a given context to answer a question. Instead of predicting labels, QA models predict the start and end positions of the answer within the context. This makes QA a span prediction problem rather than a classification problem.

2. 2. Implementation

2.1 Environment Setup

```
!pip install transformers datasets evaluate
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packag
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/di
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/pyt
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/pyt
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/pyth
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/di
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-pa
```

```
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pack
Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 8.7 MB/s eta 0:00:00
Installing collected packages: evaluate
Successfully installed evaluate-0.4.6
```

2.2 Load and Explore Dataset

```
from datasets import load_dataset

dataset = load_dataset("squad")

# To watch
dataset["train"][0]
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserW
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
README.md:       7.62k/? [00:00<00:00, 471kB/s]
plain_text/train-00000-of-00001.parquet: 100%  14.5M/14.5M [00:01<00:00, 72.3MB/s]
plain_text/validation-00000-of-00001.par(…): 100%  1.82M/1.82M [00:00<00:00, 9.10MB/s]
Generating train split: 100%  87599/87599 [00:00<00:00, 253706.14 examples/s]
Generating validation split: 100%  10570/10570 [00:00<00:00, 199983.73 examples/s]
{'id': '5733be284776f41900661182',
 'title': 'University_of_Notre_Dame',
 'context': 'Architecturally, the school has a Catholic character. Atop the
Main Building\'s gold dome is a golden statue of the Virgin Mary. Immediately
in front of the Main Building and facing it, is a copper statue of Christ with
```

2.3 BERT Model Setup

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForQuestionAnswering.from_pretrained("bert-base-uncased")
```

```
config.json: 100%                                          570/570 [00:00<00:00, 58.5kB/s]

tokenizer_config.json: 100%                                48.0/48.0 [00:00<00:00, 1.61kB/s]

vocab.txt:        232k/? [00:00<00:00, 990kB/s]

tokenizer.json:        466k/? [00:00<00:00, 1.09MB/s]

model.safetensors: 100%                                    440M/440M [00:02<00:00, 328MB/s]

Loading weights: 100%  197/197 [00:00<00:00, 504.13it/s, Materializing param=bert.encoder.layer.11

BertForQuestionAnswering LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
bert.pooler.dense.bias                      | UNEXPECTED |
bert.pooler.dense.weight                    | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
qa_outputs.weight                           | MISSING    |
qa_outputs.bias                             | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; r
- MISSING   :those params were newly initialized because missing from the checkp
```

2.4 Tokenization and Preprocessing

```
def preprocess_function(examples):
    questions = [q.strip() for q in examples["question"]]

    inputs = tokenizer(
        questions,
        examples["context"],
        max_length=384,
        truncation="only_second",
        stride=128,
        return_offsets_mapping=True,
        padding="max_length",
    )

    offset_mapping = inputs["offset_mapping"]
    answers = examples["answers"]

    start_positions = []
    end_positions = []

    for i, offsets in enumerate(offset_mapping):
        # Find the start and end of the context in the tokenized sequence
        sequence_ids = inputs.sequence_ids(i)
        context_start_token = None
        context_end_token = None
        for k, sid in enumerate(sequence_ids):
            if sid == 1: # Part of the context
                if context_start_token is None:
                    context_start_token = k
                context_end_token = k

        if context_start_token is None or context_end_token is None: # No conte
            start_positions.append(0)
            end_positions.append(0)
            continue
```

```
        answer = answers[i]
        # Handle cases where answer['text'] might be empty or missing
        if not answer["text"] or not answer["text"][0]:
            start_positions.append(0)
            end_positions.append(0)
            continue

        start_char = answer["answer_start"][0]
        end_char = start_char + len(answer["text"][0])

        # If the answer is not fully contained in the context, label it as (0,
        if not (offsets[context_start_token][0] <= start_char and offsets[conte
            start_positions.append(0)
            end_positions.append(0)
        else:
            # Otherwise, find the start and end token indices of the answer
            token_start_index = context_start_token
            while token_start_index <= context_end_token and offsets[token_star
                token_start_index += 1
            start_positions.append(token_start_index - 1)

            token_end_index = context_end_token
            while token_end_index >= context_start_token and offsets[token_end_
                token_end_index -= 1
            end_positions.append(token_end_index + 1)

    inputs["start_positions"] = start_positions
    inputs["end_positions"] = end_positions

    return inputs
```

2.5 Apply Preprocessing (Optimized for Speed)

```
tokenized_datasets = dataset.map(preprocess_function, batched=True)

#  Speed optimization
train_dataset = tokenized_datasets["train"].select(range(10000))
valid_dataset = tokenized_datasets["validation"].select(range(2000))
```

```
Map: 100%                                    87599/87599 [01:38<00:00, 807.77 examples/s]

Map: 100%                                    10570/10570 [00:11<00:00, 817.11 examples/s]
```

2.6 Fine-Tuning the Model

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="qa_finetuned",
    save_strategy="no",
    learning_rate=3e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    logging_steps=200,
    report_to="none",
    fp16=True if __import__("torch").cuda.is_available() else False,
)

trainer = Trainer(
    model=model,
    args=training_args,
```

```
    train_dataset=train_dataset,
    eval_dataset=valid_dataset,
)

trainer.train()
```

[1250/1250 03:42, Epoch 1/1]

Step	Training Loss
200	3.311210
400	1.894229
600	1.611608
800	1.379748
1000	1.355876
1200	1.287162

```
TrainOutput(global_step=1250, training_loss=1.783411752319336, metrics=
{'train_runtime': 223.8913, 'train_samples_per_second': 44.665,
'train_steps_per_second': 5.583, 'total_flos': 1959725675520000.0,
```

2.7 Evaluation (EM & F1)

```
import evaluate
import numpy as np

metric = evaluate.load("squad")

pred = trainer.predict(valid_dataset)
start_logits, end_logits = pred.predictions

raw_valid = dataset["validation"].select(range(2000))

predictions = []
references = []

for i in range(len(start_logits)):
    start = np.argmax(start_logits[i])
    end = np.argmax(end_logits[i]) + 1

    offsets = valid_dataset[i]["offset_mapping"]
    context = raw_valid[i]["context"]

    if start < len(offsets) and end <= len(offsets):
        start_char = offsets[start][0]
        end_char = offsets[end - 1][1]
        answer = context[start_char:end_char]
    else:
        answer = ""

    predictions.append({"id": raw_valid[i]["id"], "prediction_text": answer})
    references.append({"id": raw_valid[i]["id"], "answers": raw_valid[i]["answe

score = metric.compute(predictions=predictions, references=references)

print(score)
```

Downloading builder script: 4.53k/? [00:00<00:00, 82.5kB/s]

Downloading extra modules: 3.32k/? [00:00<00:00, 175kB/s]

```
{'exact_match': 63.55, 'f1': 72.10013892343385}
```

2.8 Custom Testing

```
import torch

def get_answer(question, context):
    inputs = tokenizer(question, context, return_tensors="pt")

    # Move inputs to the same device as the model
    inputs = {k: v.to(model.device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model(**inputs)

    start = torch.argmax(outputs.start_logits)
    end = torch.argmax(outputs.end_logits) + 1

    return tokenizer.convert_tokens_to_string(
        tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][start:end])
    )

# Example 1
print(get_answer(
    "Who developed the theory of relativity?",
    "Albert Einstein developed the theory of relativity."
))

# Example 2
print(get_answer(
    "Where is the Eiffel Tower located?",
    "The Eiffel Tower is located in Paris, France."
))

albert einstein
paris, france
```

3. Results

The fine-tuned model achieved the following performance on the validation dataset:

Exact Match (EM): 63.55% F1 Score: 72.10% Custom Test Outputs: Q1: Who developed the theory of relativity? → Albert Einstein Q2: Where is the Eiffel Tower located? → Paris, France

The higher F1 score compared to EM indicates that the model often predicts partially correct answers even when the exact span is not perfectly matched.

4. Reflection

Fine-tuning a transformer model for question answering provided valuable insights into how modern NLP systems operate beyond traditional classification tasks. Unlike classification, QA requires predicting exact spans within a context, which makes preprocessing more complex. One of the main challenges was correctly aligning answer spans with tokenized inputs. I also learned how powerful pre-trained models like BERT can be when adapted to specific tasks. The Hugging Face ecosystem greatly simplified model training and evaluation. Overall, this assignment enhanced my understanding of transformer-based architectures and their real-world applications.

Thank You

Forkan Amin

AI Engineering